Chapter 1

An Optimal LZ4 Encoder: How Much Smaller Can LZ4 Get?

Author: Keisuke Goto

This article is an English translation of the Japanese article "LZ4 最適エンコーダ LZ4 はあとどれだけ小さくできるのか," originally published in 甲乙丙技術部 Vol. 1 (Techbookfest 21, November 2026), available at `https://techbookfest.org/product/paqm1HiaKWmenVnAF8cNsQ`. The translation was prepared with the assistance of machine translation. In case of any discrepancy, the Japanese original takes precedence.

1.1 Introduction

How much smaller can data compressed with LZ4[1] still get? In this article, we propose LZ4opt, an optimal algorithm that minimizes the compressed size while conforming to the LZ4 compression format[*1]. By comparing the compressed sizes of LZ4opt and the reference implementation, we explore how much room for improvement remains in the compression ratio.

LZ4 is a compression method developed by Yann Collet and is characterized by fast compression and decompression. It builds on LZ76[9], a compression method proposed by Abraham Lempel and Jacob Ziv, and its derivative LZ77[10]. These two methods compress redundant data by representing strings that occur repeatedly

[*1] In this article, "optimal" means achieving this minimum size.

in the input (matches) as references to their previous occurrences. LZ4 splits the input into blocks and, in each block, uses hashing to find match candidates quickly. In addition, it achieves fast compression and decompression by not using entropy coding and instead encoding matches and strings stored as is simply at the byte level. Because of these characteristics, LZ4 is used in a wide range of applications, including the compression of data in storage, in communication, and in memory. For example, it is used for file system compression in OpenZFS[5] and for column value compression in PostgreSQL[8]. Other examples include the compression of intermediate data in Apache Spark[7], AssetBundles in Unity[6], network traffic in online games[2], and RPC messages[3]. Linux' s zram also uses it to reduce memory usage by compressing swap space and other data[4].

LZ4 has been improved for more than 10 years since its release in 2011. However, it is not clear how close the compression ratio of existing implementations is to the optimum. To investigate this gap, we need to compute the minimum size achievable in the LZ4 format and compare it with the compressed sizes produced by existing implementations.

A clue comes from LZ4HC, the high-compression mode of the reference implementation. At its higher compression levels, LZ4HC uses dynamic programming to reduce the compressed size, and behind this lies the optimization idea of finding a shortest path in a graph[*2]. However, neither this idea nor the details of the algorithm are documented, so understanding its behavior and verifying its optimality requires reading the source code. In addition, the implementation omits some searches and comparisons to reduce computation, so the minimum size is not guaranteed. If we remove these shortcuts and optimize exactly, the computational cost of the search and the dynamic programming becomes a problem instead.

In this article, we organize the optimization ideas behind LZ4HC and, based on them, propose LZ4opt, an algorithm that computes the minimum size in linear time.

- **Explanation of the LZ4 algorithms**: Based on the reference implementation, we explain the algorithms of LZ4' s fast mode (the default) and high-compression mode (LZ4HC).
- **Formulation of the compressed-size minimization problem**: Within the LZ4 format, we formulate the minimization of the compressed size as a shortest

[*2] The connection to shortest paths is not stated explicitly in the reference implementation or its documentation; it is our interpretation, based on reading the source code and understanding its behavior.

path problem on a graph.

- **A linear-time optimal algorithm**: We propose LZ4opt, which computes the minimum size in time linear in the block length.
- **Implementation and evaluation**: We implement LZ4opt and compare its compression ratio and speed with those of the reference implementation.

In the rest of this article, starting from LZ76 and LZ77, we describe the LZ4 format and the reference implementation and formulate the compressed-size minimization problem. We then present LZ4opt and experimentally evaluate how its compression ratio and speed differ from those of the reference implementation. Throughout this article, we assume that the alphabet size is a constant. Since LZ4 processes data byte by byte, the alphabet size is 256.

1.2 LZ76

For a string $T = t_1 t_2 \cdots t_n$ of length n, among the substrings starting at position p, the longest one that also starts at some position before p is called the Longest Previous Factor (LPF), and we define its length as $lpf(p)$.

$$lpf(p) = \max\big(\{\ell \mid 1 \le \ell \le n - p + 1,\ \exists q < p,\ T[q..q+\ell-1] = T[p..p+\ell-1]\} \cup \{0\}\big)$$

LZ76 factorizes T as $T = f_1 \dots f_z$, where the factor f_i starting at position p_i consists of the LPF for p_i followed by one character[*3]. Figure 1.1 shows the factorization of $T = \texttt{aabaabaabb}$. f_2 is an LPF of length 1 matching $T[1..1]$ followed by `b`, and f_3 is an LPF of length 6 matching $T[1..6]$ followed by `b`. As with f_3, a previous occurrence of the LPF may extend across the starting position $p_3 = 4$.

[*3] If the LPF reaches the end of the string, the entire remaining string becomes the last factor.

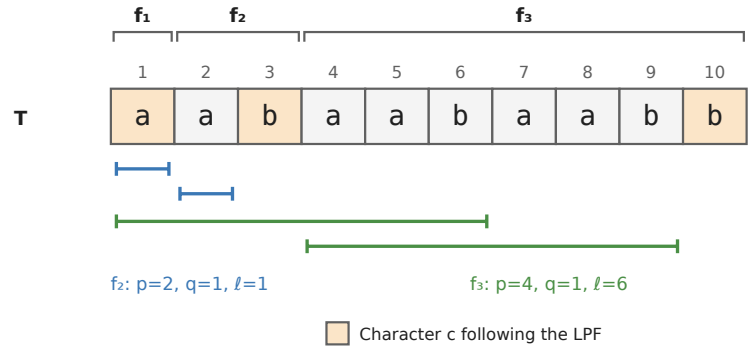


Figure 1.1: Factorization of the string $T = \texttt{aabaabaabb}$ by LZ76

Here, letting q_i be the position of the previous occurrence of the LPF, $\ell_i = lpf(p_i)$ its length, and c_i the character following it, f_i can be represented by the triple (q_i, ℓ_i, c_i), and the original f_i can be restored by referring to T. LZ76 can thus be viewed both as a string factorization method and as a compression algorithm.

$$\begin{aligned} f_i &= T[p_i .. \min(n, p_i + \ell_i)] \\ &= T[q_i .. q_i + \ell_i - 1]c_i \end{aligned}$$

1.3 LZ77

LZ77 is LZ76 with the starting position of the match source restricted, when searching for the LPF, to the preceding w characters (the window). If we define $lpf_w(p)$ as follows, LZ77 can be viewed as LZ76 with lpf in the factorization replaced by lpf_w.

$$\begin{aligned} lpf_w(p) = \max\big(\{\ell \mid 1 \le \ell \le n - p + 1,\ \exists q,\ \max(1, p - w) \le q < p, \\ T[q..q + \ell - 1] = T[p..p + \ell - 1]\} \cup \{0\}\big) \end{aligned}$$

Instead of the starting position q_i of the match source, LZ77 uses the offset $d_i = p_i - q_i$ from the current position p_i. The factor f_i can be represented by the triple (d_i, ℓ_i, c_i) of the offset, the match length $\ell_i = lpf_w(p_i)$, and the character c_i following the LPF. When $\ell_i = 0$, there is no match source, so we use $(0, 0, c_i)$. Since the window guarantees $1 \le d_i \le w$, the offset can be represented in $\lceil \log_2 w \rceil$ bits. The match search can also be limited to at most w positions.

To save computation time, many compression algorithms based on LZ77 do not compute the LPF exactly and instead approximately find as long a match as possible. For simplicity, this article also calls such an approximately found match an LPF and

states explicitly when it needs to be distinguished from the exact LPF. Hereafter, unless otherwise noted, we also write *lpf* for the window-restricted lpf_w.

1.4 LZ4

LZ4 divides the input string into blocks of a fixed size and compresses each block separately. The compressed result of each block is a series of pairs of "literals (an uncompressed string stored as is) + a match (a reference to a previous occurrence, represented by an offset and a match length)." The LZ4 specification calls this pair a *sequence*, which corresponds to a factor in LZ77. Hereafter, following the specification, we call it a sequence when describing the LZ4 format, and a factor when discussing factorizations alongside LZ76/77.

Below, we first explain the relationship between blocks and frames, and then the byte layout of a sequence, which makes up the contents of a block, and its encoded length. After that, we describe the basic strategy by which an LZ4 encoder builds the series of sequences.

Blocks and Frames

The block size is chosen from 64KiB, 256KiB, 1MiB, and 4MiB (the default of the `lz4` command is 4MiB blocks, which we also use in our experiments)[*4]. A block by itself carries no information about its length or type; the enclosing LZ4 frame records the compressed size of each block and whether it is compressed. In addition, if compressing a block does not make it smaller, the frame stores the block uncompressed (distinguished by the most significant bit of the size field). There are two kinds of blocks: independent blocks and dependent blocks. In an independent block, matches are searched only within that block, so each block can be decompressed on its own. In a dependent block, the last part of the previously decompressed data (up to 64KiB) can also be used for match candidates, which can improve the compression ratio, but decompression requires the preceding blocks. Blocks are independent by default, and this article also assumes independent blocks.

[*4] This value is the maximum block size, and the specification does not require the input to be split into blocks of exactly this size. However, the reference implementation splits the input at this size, so only the last block is shorter. This article likewise assumes that the input is split at this size and that only the last block is shorter.

Byte Layout of a Sequence

Figure 1.2 shows the byte layout of a sequence.

- Token (1 byte): holds part of both the literal length and the match length
- Literal length extension bytes (only when needed)
- Literal bytes
- Offset (2 bytes)
- Match length extension bytes (only when needed)

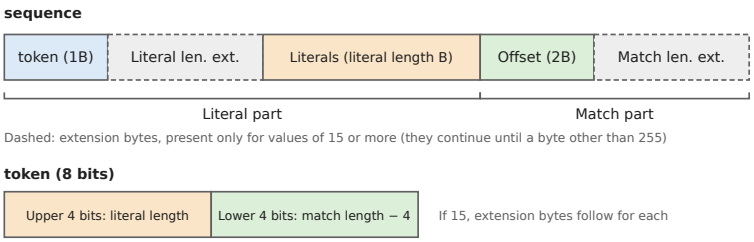


Figure 1.2: Byte layout of an LZ4 sequence

The token is 1 byte; its upper 4 bits represent the literal length, and its lower 4 bits represent "match length − 4." The 4 is subtracted because the minimum match length of LZ4 is 4.

Since 4 bits can represent values from 0 to 15, longer lengths are represented with extension bytes. When the corresponding bits of the token are 15, extension bytes follow immediately, and their values are added up until a byte other than 255 appears (including that byte). For example, when the literal length is 300, the upper 4 bits of the token are 15 and the extension bytes are $255, 30$, representing $15+255+30 = 300$. If the literal length is 14 or less, no extension bytes are needed, and the 1-byte token alone suffices. The same applies to the match length: when the lower 4 bits are 15, extension bytes follow the offset. The encoding itself places no upper limit on the match length, but a match must fit within the block.

The literal bytes store, as is, a byte string of the length specified by the literal length. The literal length is 0 or more; when it is 0, the literals are empty and another match immediately follows the previous match. The offset is then stored in 2 bytes in little-endian order. The offset takes a value from 1 to 65535. In other words,

the window size for computing the LPF is fixed at $w = 65535$ (about 64KiB).

Decompression works as in LZ77: after outputting the literals, the decoder copies match-length bytes, one byte at a time, from the position that lies offset bytes back. When the offset is smaller than the match length, the copy source overlaps the region being decompressed, and the same pattern repeats.

Only the last sequence of a block has no match and ends with just a token and literals. Therefore, the lower 4 bits of the token of the last sequence are unused. In addition, the format requires that the last 5 bytes of a block always be literals and that the last match start at least 12 bytes before the end of the block (so that decompression can safely copy in wide units).

Letting a be the literal length and m the match length, the number of extension bytes is $\lfloor (a - 15)/255 \rfloor + 1$ when $a \geq 15$ and 0 when $a < 15$. The same applies to the match length, using $m - 4$. Writing $e(x)$ for the number of extension bytes, the encoded length of a sequence is given by the following expression.

$$1 + e(a) + a + 2 + e(m - 4)$$

Using a match requires at least 3 bytes for the token and the offset, so a match of length 3 or less takes as many bytes as storing the original string as literals, or more. Therefore, setting the minimum match length to 4 is a natural design.

Basic Compression Strategy of LZ4

The fast mode of the reference implementation (the default of the `lz4` command) is based on a greedy strategy. It scans the input from the beginning, and if it finds a match of 4 or more characters at a position, it extends the match as far as possible and adopts it. If it finds no match of 4 or more characters, it makes the character at that position a literal and moves on to the next position.

Matches are searched using a hash table. In fast mode, the key is the hash value of the few bytes starting at the current position (usually 5 bytes on 64-bit platforms and 4 bytes on 32-bit platforms), and the table records only the last position with that hash value. At each position, a match candidate is taken from the table. If the first 4 bytes match those of the candidate and the offset is 65535 or less, the match is extended and output as one sequence together with the preceding literals.

Because only one candidate is kept, older positions are overwritten and lost, and a longer match may be missed even if one exists. In addition, while no match is found, the search step size is gradually widened. For the first 64 attempts, the current

position advances 1 byte at a time; after that, the step size grows by 1 byte, to 2, 3, ⋯ bytes, after every 64 consecutive failures. When a match is found, the step size is reset to 1. Matches starting at skipped positions are not searched, which can lower the compression ratio, but even for nearly random data the hash table does not need to be consulted at every position.

Greedy parsing, which keeps taking the longest match (LPF) at each position, minimizes the number of factors in LZ76/LZ77[9],[10]. However, minimizing the number of factors does not necessarily minimize the encoded length. This is because in LZ4 a literal costs 1 byte per character, whereas each match costs at least 3 bytes. In (a) of Figure 1.3, the longest matches of length 6 at position p and of length 25 at position $p+6$ are taken, and the two sequences take 7 bytes in total. In contrast, (b) makes the character at position p a literal and takes the longest match of length 30 at position $p+1$, for a total of 5 bytes.

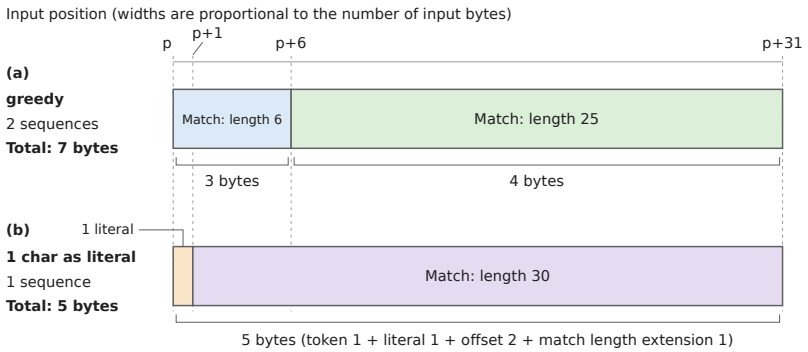


Figure 1.3: Comparison of greedy parsing and a parsing with a smaller encoded length

When the compression ratio matters, the high-compression mode (*LZ4HC*) can be used. The `lz4` command uses it at level 3 and above (`-3` to `-12`). Its output has the same format as that of fast mode, so it can be decompressed in the same way. LZ4HC improves the compression ratio by following multiple candidates per hash value as a chain and searching for longer matches. We explain the details in the "LZ4HC" section below.

1.5 Formulating Compressed-Size Minimization for LZ4

Compressed-size minimization for LZ4 can be formulated as a shortest path problem by representing a parsing of the input as a path in a graph. Levels 10 to 12 of LZ4HC also use dynamic programming based on this idea.

Whether we choose a literal at a position, or which match length we choose, changes the matches available afterward and the overall encoded length. To minimize the encoded length, we need to compare choices while taking the subsequent parsing into account. We therefore represent a parsing of the input as a path in a graph. Each vertex is a pair (i, a) of an input position i and the length a of the pending literals at that position. From each vertex, we draw an edge to $(i + 1, a + 1)$ that adds one literal character, and an edge to $(i + m, 0)$ that adds a match of length m ($m \geq 4$) available at that position. Adding a match also outputs the preceding literals, so the pending literal length returns to 0.

The weight of each edge is the increase in encoded length caused by that choice. Using the number of extension bytes $e(x)$ defined above, we define the cost $L(a)$ for a literal length a and the cost $M(m)$ for a match length m as follows.

$$L(a) = a + e(a), \qquad M(m) = 3 + e(m - 4)$$

$L(a)$ is the sum of the a literal bytes and the literal length extension bytes, and $M(m)$ is the sum of the token (1 byte), the offset (2 bytes), and the match length extension bytes. $L(a) + M(m)$ is the encoded length of one sequence with literal length a and match length m. In this accounting, only $L(a)$ is counted for the pending literals, and the 1 byte of the token is counted when the next match is added[*5].

Therefore, the two kinds of edges and their weights are as follows.

- Literal edge $(i, a) \to (i + 1, a + 1)$: $L(a + 1) - L(a)$ bytes.
- Match edge $(i, a) \to (i + m, 0)$ $(4 \leq m \leq lpf(i))$: $M(m)$ bytes.

The weight of a literal edge is usually 1 byte, but when $a + 1$ reaches 15, 270, 525, $\cdots$ (every 255 after 15), where the number of extension bytes increases, an extension byte is added on top of the literal byte, making the weight 2 bytes. Thus, determining

[*5] When a literal is added, more literals may follow, so the literal length of that sequence is not yet fixed. Adding a match fixes both the literal length and the match length, so counting the cost of the token, which holds both lengths, at that point keeps the computation simple.

the weight of a literal edge requires not only the input position but also the pending literal length. With vertices defined as (i, a), the weights of both kinds of edges are determined without depending on the path taken so far.

Since every edge advances the input position, this graph is a directed acyclic graph (DAG). A parsing of the input corresponds to a path from the first vertex $(0, 0)$ to a vertex at the last position. Adding the token (1 byte) of the final literal-only sequence to the total weight of the edges gives the encoded length of that parsing. This final byte is common to all paths, so the parsing that minimizes the encoded length can be found as a shortest path from the beginning to the end of this DAG.

1.6 LZ4HC

LZ4HC, the high-compression mode of LZ4, finds a parsing with a greedy strategy with lookahead at levels 3 to 9, and with a DP parser based on dynamic programming at levels 10 to 12[*6]. Below, we assume v1.10.0 of the reference implementation with standard settings, that is, without the option that favors decompression speed. Table 1.1 shows the settings for each level. The number of search attempts is the upper limit on the number of candidates examined at one position. The early-commit threshold is the match length above which a match is adopted without being compared against other parsings.

Table 1.1: LZ4HC settings by level

Level	Parsing method	Search attempts	Early-commit threshold
3 – 9	Greedy with lookahead	4 – 256	–
10	DP parser	96	64
11	DP parser	512	128
12	DP parser	16,384	4,095

The DP parser divides the input into short segments and uses a relative position i that is 0 at the start of each segment. It advances i from 0 and, based on the cost to reach position i (the encoded length) `opt`$[i]$, updates the costs to reach the positions

[*6] The reference implementation calls it the optimal parser. However, as described later, it does not necessarily minimize the encoded length of the entire block. This article therefore calls it the DP parser, focusing on its use of dynamic programming.

ahead. Below, we explain how LZ4HC omits parts of the computation compared with the formulation in the previous section, and summarize the resulting issues.

LPF computation: LZ4HC finds match candidates with a hash table and follows past candidates with the same hash value through a chain table. It chooses the longest match among the candidates examined within the window, but because the number of candidates examined is limited, the exact LPF is not necessarily obtained even at level 12.

Keeping only one state per position: A naive computation of the shortest path in the previous section would need to store the minimum cost for each vertex (i, a), that is, for each pending literal length a, even at the same position i. LZ4HC keeps at most one state per position and records its cost $\texttt{opt}[i]$ and pending literal length b_i. An update by a literal is adopted only if it lowers the cost, whereas an update by a match is adopted even at equal cost. Therefore, a state first reached by a match is not replaced even if it is later reached by a literal at the same cost. However, the cost of extending literals beyond that point can depend on the pending literal length, so this policy does not guarantee optimality[*7].

Skipping updates: At positions where LZ4HC judges, from the recorded costs to reach, that the encoded length cannot be improved, it skips the match search and the updates. At levels 10 and 11, it searches only for matches long enough to go beyond the farthest position already reachable by a match.

Limiting the processing range: The DP parser of LZ4HC finds a parsing for each short segment and commits it before moving on to the next segment. The segment size is limited by the constant `LZ4_OPT_NUM` in `lib/lz4hc.c` (default 4,096)[*8]. However, the parsing may be committed before this limit is reached, for example by an early commit of a long match. The early-commit threshold at level 12 is `LZ4_OPT_NUM` − 1 (4,095 by default).

A committed parsing cannot be revised to take later matches into account. Even if `LZ4_OPT_NUM` is enlarged so that the entire block fits, the conditions for committing midway remain, so optimality over the entire block is not guaranteed.

[*7] Consider two states that reach the same position at the same cost, with pending literal lengths 0 and 15. If 15 more literals follow, the state with length 0 needs a new extension byte, so its cost increases by 16 bytes. The state with length 15 needs no additional extension byte, so its cost increases by 15 bytes. Therefore, prioritizing the length-0 state reached by a match can lose a candidate whose cost later becomes smaller.

[*8] In our experiments, we also compare a version rebuilt with `LZ4_OPT_NUM` in the source changed to 65,536.

Computational Challenges in Finding an Optimal Solution

Finding the minimum encoded length of an entire block as a shortest path in the formulation, without restrictions like those of LZ4HC, poses the following three computational challenges.

Search cost of computing the exact LPF: If the exact LPF is computed with LZ4HC' s hash table and chain table without cutting the search short, then for inputs with many candidates sharing the same hash value, the number of candidates examined at one position can approach the window size of 65,535. Since string comparisons are also needed to compute the match length with each candidate, searching at every position is expensive.

Growth in the number of states due to the pending literal length: The vertices of the formulated graph are pairs (i, a) of a position i and a pending literal length a. Since $0 \leq a \leq i$, there are up to $i + 1$ states at position i and $O(n^2)$ states over the entire block in the worst case, which requires memory and computation time to store the costs and explore the edges. We need to reduce the number of states without discarding candidates that lead to an optimal parsing.

Growth in the number of updates per match length: Even if the above two challenges are solved, the number of updates by matches remains a problem. If matches of each length from 4 to $lpf(i)$ are tried individually and the costs to reach the positions ahead are updated based on $\texttt{opt}[i]$, the number of updates from position i is $\max(0, lpf(i) - 3)$. For inputs with many long matches, the total number of updates over the entire block is $O(n^2)$ in the worst case.

LZ4opt, described in the next section, finds the minimum encoded length of an entire block while keeping the search cost, the number of states, and the number of updates small.

1.7 LZ4opt

LZ4opt, which we propose in this article, is an encoder that, like LZ4HC, finds a parsing with dynamic programming, and that guarantees the minimum encoded length for each block. It addresses the challenges of search cost, number of states, and number of updates identified in the previous section with the following methods, respectively.

- **Search cost of computing the exact LPF**: Instead of enumerating candi-

dates with hash chains, it uses a sliding suffix tree to compute the exact LPF quickly.

- **Growth in the number of states due to the pending literal length**: It compares the cost to reach and the headroom before the literal length requires another extension byte, and reduces the states kept at each position to one while preserving optimality.
- **Growth in the number of updates per match length**: Instead of updating individually for each match length, it computes at once, for each destination, the minimum cost among the match candidates that reach it.

Unlike LZ4HC, it does not commit the parsing segment by segment but targets the entire block. It neither skips positions to update nor commits long matches early, and it finds the shortest path in the formulated graph. Below, we explain the three methods in turn.

Fast Computation of the Exact LPF

LZ4opt computes the exact LPF using a sliding suffix tree. A sliding suffix tree[13],[14] is an index structure that updates a suffix tree[11],[12] online over a sliding window. The LPF at position p can be computed in $O(lpf(p))$ time, proportional to its length and independent of the window size.[*9] Moreover, when the LPFs are computed in order from the beginning, earlier computations can be reused, so the LPF at each position can be computed in $O(1)$ amortized time, and the LPFs of an entire block of length n in $O(n)$ time.

State Reduction That Preserves Optimality

Like LZ4HC, LZ4opt keeps only one state per position and records its cost $\texttt{opt}[i]$ and pending literal length b_i. The candidates reaching position i are of the following two kinds, each represented as a pair of a cost and a literal length.

- Literal: the candidate obtained by extending the literals from position $i-1$ by one character, with cost $\texttt{opt}[i-1] + L(b_{i-1}+1) - L(b_{i-1})$ and literal length $b_{i-1}+1$
- Match: a candidate reached by a match of length $i-j$ from a starting position

[*9] With alphabet size σ, computing the LPF takes $O(lpf(p)\log\sigma)$ time. In LZ4, $\sigma = 256$ is a constant, so we omit the dependence on the alphabet size in the main text.

j satisfying $4 \le i - j \le lpf(j)$, with cost $\texttt{opt}[j] + M(i-j)$ and literal length 0

Among these candidates, we keep the one with the smaller cost, and if the costs are equal, the one with the larger headroom before the next extension byte (the number of literal characters that can be added without adding an extension byte). We set $\texttt{opt}[i]$ and b_i to the cost and literal length of the kept candidate, and also record the edge used to reach it so that the parsing can be restored.

This selection preserves optimality for the following reason. Placing a match resets the pending literal length to 0, so the only way in which candidates can differ in their effect on later costs is through the extension bytes added when the literals are extended. When the literals are extended by k characters, the number of added extension bytes is, for any candidate, either $\lfloor k/255 \rfloor$ or one more than that. Since costs are integers in bytes, a candidate with a smaller cost never exceeds the cost of another candidate, even after the literals are extended. When the costs are equal, the candidate with the larger headroom before the next extension byte gains extension bytes more slowly, so its cost never exceeds that of the other candidate. Therefore, keeping only one state per position under this rule does not lose the minimum encoded length.

Batched Updates over Match Lengths

An update by a literal only extends the state at the previous position by one character, so it takes $O(1)$ time per position. Updates by matches, if every match length is tried from each starting position, take $O(n^2)$ time over the entire block in the worst case. LZ4opt changes the viewpoint: it fixes the end position j of a match and finds the minimum cost by comparing the candidate starting positions i that reach it. By exploiting the way the encoded length grows, it narrows the candidates to compare down to a constant number and, while preserving the minimum cost, processes the updates at each position in amortized $O(1)$ time and the entire block in $O(n)$ time.

The positions i that reach j are those satisfying $i \le j - 4$ and $i + lpf(i) \ge j$. Since $i + lpf(i)$ is non-decreasing in i[*10], the candidates form a sliding window whose ends both move to the right as j advances. The minimum of $\texttt{opt}[i] + M(j-i)$ within this window is the minimum cost to reach j by a match.

$M(j-i)$ is constant at 3 for match lengths up to 18, and for lengths of 19 or more

[*10] Shifting a match at position i by one character still gives a match of at least $lpf(i)-1$ characters at position $i+1$. Therefore, if $i < i' < i+lpf(i)$, then $i'+lpf(i') \ge i+lpf(i)$, and if $i' \ge i+lpf(i)$, the claim is trivial since $lpf(i') \ge 0$.

it increases by 1 every 255 characters. We therefore keep, for short matches, only the candidate with the smallest `opt`$[i]$, and for long matches, we divide the candidates into 255 classes by the remainder of i divided by 255 and keep the minimum-cost candidate of each class. Candidates in the same class gain extension bytes at the same time, so their order by cost does not change as j advances. As a result, the match candidates compared at each position are narrowed down to at most 256: one for short matches and one minimum-cost candidate for each class of long matches.

Each class is managed with a monotonic queue. A monotonic queue keeps the candidates in order of starting position while maintaining costs that decrease monotonically from the tail toward the head. When a new candidate is added, removing from the tail the candidates whose cost is at least that of the new candidate, and removing from the head the candidates that have left the window, keeps the minimum-cost candidate always at the head. Each candidate is added and removed at most once, so maintaining the queues takes $O(n)$ time in total. We take the minimum-cost candidate of each class from the head of its queue in $O(1)$ time each, and compare these at most 256 candidates and the candidate that extends the literals by one character, using the state selection rule described above. This gives the same minimum cost as examining all matches.

The following pseudocode, written in Python-like notation, shows the overall procedure. The LPF lengths are precomputed and stored in an array, which is written as `LPF`$[i]$ in the pseudocode. A state is represented as a tuple `(cost, literal_length, previous)`, where `cost` is the encoded length up to that position, `literal_length` is the length of the pending literals, and `previous` is the preceding position used to restore the parsing. `choose_state` is a function that, following the state selection rule described above, chooses the state with the smaller cost, or, if the costs are equal, the state with the larger headroom before the next extension byte.

```
# Encoded length up to position j when positions i to j form one match
def match_cost(i, j):
    return opt[i] + M(j - i)

# Push i to the queue at j; pop tail candidates costing >= i (monotonic)
def push_candidate(queue, i, j):
    while queue and match_cost(queue[-1], j) >= match_cost(i, j):
        queue.pop()
    queue.append(i)

# Remove candidates no longer usable at position j from the queue head
def expire(queue, j, is_short):
```

```
    # max_length is the maximum match length handled by the queue
    # 18 for the short queue (lengths 4-18); 19+ go to the long queues
    # The long queues have no upper limit, so use infinity
    max_length = 18 if is_short else inf
    while queue:
        i = queue[0]
        deadline = i + min(max_length, LPF[i])
        if j <= deadline:
            break
        queue.popleft()

short_queue = deque()  # candidates of length 4-18
long_queues = [deque() for _ in range(255)]  # length 19+, by start mod 255
queues = [short_queue] + long_queues  # all queues compared for min cost
opt[0] = 0  # cost of the state chosen at each position
b[0] = 0  # pending literal length of the state chosen at each position

for j in range(1, n + 1):
    # Remove candidates expired at position j from the head of each queue
    expire(short_queue, j, is_short=True)
    for queue in long_queues:
        expire(queue, j, is_short=False)

    # Add the candidate that has just reached length 4 to the short queue
    i = j - 4
    if i >= 0 and LPF[i] >= 4:
        push_candidate(short_queue, i, j)

    # Add the candidate that has just reached length 19 to its long queue
    # Added independently: may be needed even if expired from short queue
    i = j - 19
    if i >= 0 and LPF[i] >= 19:
        push_candidate(long_queues[i % 255], i, j)

    # Start from the previous state extended by one literal character
    literal_length = b[j-1] + 1
    literal_cost = opt[j-1] + L(literal_length) - L(b[j-1])
    best = (literal_cost, literal_length, j-1)

    # The head of each queue gives the minimum cost of its class
    for queue in queues:
        if queue:
            i = queue[0]
            candidate = (match_cost(i, j), 0, i)
```

```
            # On a tie, keep the state with more headroom
            best = choose_state(best, candidate)

    # Save the best state and the previous position to restore the parsing
    opt[j], b[j], previous[j] = best
```

In this pseudocode, at each destination position, we check all 256 queues for expired candidates and compare the head candidates of the queues to find the minimum cost. Since the number of classes is constant, the total time is $O(n)$, but the LZ4opt implementation avoids this scan to improve practical speed. For the expiry check, it uses the non-decreasing property of $i + lpf(i)$ to process, in order of starting position, only the queues of candidates that expire, instead of examining all 256 queues. For selecting the minimum cost, it manages the head candidates of the long-match classes with a segment tree and finds the minimum-cost candidate without scanning the 255 classes every time[*11].

The updates take amortized $O(1)$ time per position for the queue operations, and $O(n)$ time for the entire block. By combining exact LPF computation, state reduction, and batched updates, LZ4opt finds the minimum encoded length of an entire block in $O(n)$ time. Since it stores, for each position, the cost and the information needed to restore the parsing, its memory usage is $O(n)$. Its output conforms to the LZ4 format, so it can be decompressed with the official `lz4`.

1.8 Experiments

We compare the reference implementation of LZ4 and LZ4opt, which we implemented in Rust[*12], in terms of compression ratio and computation speed. We compared the following six encoders.

[*11] The tree stores $\texttt{opt}[i] - \lfloor i/255 \rfloor$, which does not depend on the destination position j, together with the starting position i. The correction to the actual cost takes only two values, which differ by 1 byte depending on the class, so we split the classes into two ranges, query the minimum of each range, apply the corrections, and compare the results.

[*12] Repository of the LZ4opt implementation: `https://github.com/kg86/lz4opt`.

Encoder	Description
LZ4 default	Default of the official CLI (level 1), using the greedy strategy
LZ4HC -9	Default level of the LZ4HC library; a greedy strategy with lookahead that searches for candidates by following hash chains
LZ4HC -10	Lowest level that uses the DP parser
LZ4HC -12	Highest level that uses the DP parser
LZ4HC -12w	LZ4HC -12 built with `LZ4_OPT_NUM` changed from 4,096 to 65,536 (w stands for wide)
LZ4opt	An optimization strategy that minimizes the encoded length of the entire block based on the exact LPF

The main point of comparison for LZ4opt is LZ4HC -12, the highest-compression setting available in the official distribution. LZ4HC -12w is a setting not available in the official distribution; we added it to check whether the difference between LZ4HC -12 and LZ4opt comes from the limited optimization range. For the sliding suffix tree in LZ4opt, we used "snakree"[*13], an implementation of the algorithm by Leonard et al.[15].

For evaluation, we used the Silesia corpus[16] (12 files, about 202MiB in total), which is also used in LZ4' s official benchmarks, and the Pizza&Chili repetitive corpus[17] (29 files, about 5.3GiB in total). All encoders were run with a single thread and produced LZ4 frames in the independent block format with a block size of 4MiB. LZ4opt splits blocks in the same way as the reference implementation and optimizes only the parsing within each block. All outputs were decompressed with the official `lz4`, and we confirmed that they matched the original files.

The experiments were run on an Apple M4 (32GiB of memory, macOS 27.2), using LZ4 1.10.0 for the reference implementation and a release build with Rust 1.99.1 for our implementation. The running time was measured from process start to exit; for each file, we used the median of 3 measurements for the Silesia corpus and a single measurement for the Pizza&Chili corpus, which takes longer to measure. The speed for an entire corpus is the total input size divided by the sum of the running times used for each file. The maximum memory is the largest of the peak RSS values (the maximum physical memory usage) recorded in all measurements of all files.

[*13] Repository of the sliding suffix tree implementation: `https://github.com/kg86/snakree`.

Results on the Silesia Corpus

Table 1.2 shows the results for the entire Silesia corpus, and Figure 1.4 shows the per-file results, with speed on the horizontal axis and compression ratio on the vertical axis. The compression ratio is the original size divided by the compressed size; larger values indicate better compression. The ratio difference is the compression ratio of each method minus that of LZ4opt.

Table 1.2: Overall comparison on the Silesia corpus

Encoder	Compressed size (bytes)	Compression ratio	Ratio difference	Compression speed (MiB/s)	Max RSS (MiB)
LZ4 default	100,934,427	2.100	− 0.639	631.0	14.0
LZ4HC -9	77,992,601	2.717	− 0.022	48.9	14.0
LZ4HC -10	77,704,815	2.727	− 0.012	32.8	14.0
LZ4HC -12	77,372,720	2.739	− 0.00001	15.8	14.2
LZ4HC -12w	77,372,450	2.739	− 0.0000004	16.5	14.2
LZ4opt	77,372,438	2.739	0	4.4	359.9

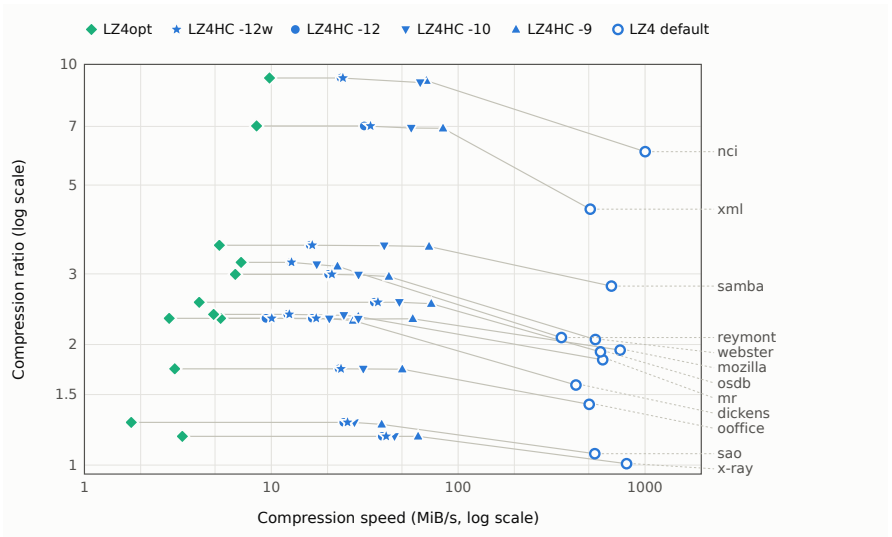


Figure 1.4: Compression speed and compression ratio for each file in the Silesia corpus

Each point in the figure represents the result of compressing one file with one encoder. The points for the same file are connected by lines in the order LZ4opt, LZ4HC -12w, -12, -10, -9, and LZ4 default.

As the official level increases, the points generally move from right to left, confirming the tendency for the compression ratio to increase at the cost of speed. The change from LZ4 default to LZ4HC -9 is the largest, and the change from LZ4HC -9 to LZ4HC -12 is smaller. For all files, the points of LZ4opt are at almost the same height as those of LZ4HC -12 and to their left, showing that the compression ratio is almost unchanged while the speed is lower. The points of LZ4HC -12w also nearly overlap those of LZ4HC -12, so widening the search range barely changes the compression ratio.

The difference between LZ4opt and LZ4HC -12, the highest official compression setting, was only 282 bytes over the entire input of about 202MiB. Looking at individual files, the compressed sizes of the two matched exactly for 6 of the 12 files, and the difference was at most 98 bytes for the remaining 6 files. In other words, on the Silesia corpus, LZ4HC -12 almost reaches the minimum size achievable under this block splitting. With LZ4HC -12w, the difference from LZ4opt shrank further to 12 bytes overall, and the sizes matched exactly for 9 files. In terms of speed, LZ4HC -12 ran at 15.8MiB/s whereas LZ4opt ran at 4.4MiB/s, taking about 3.6 times as long to compress. The maximum RSS was 359.9MiB for LZ4opt, compared with 14.2MiB for LZ4HC -12. Because LZ4opt maintains the tree structure of the sliding suffix tree, its memory usage, although linear, has a large constant factor, so it consumes a lot of memory.

Results on the Highly Repetitive Pizza&Chili Corpus

Table 1.3 shows the results for the entire Pizza&Chili corpus, a collection of highly repetitive strings. Over the entire corpus, the difference in compressed size between LZ4opt and LZ4HC -12 was 425,963 bytes and the difference in compression ratio was a small 0.003, so, as with the Silesia corpus, their compression ratios are almost the same.

Table 1.3: Overall comparison on the Pizza&Chili corpus

Encoder	Compressed size (bytes)	Compression ratio	Ratio difference	Compression speed (MiB/s)	Max RSS (MiB)
LZ4 default	1,620,776,282	3.520	− 2.447	1,004.4	13.2
LZ4HC -9	992,256,398	5.750	− 0.217	21.8	13.5
LZ4HC -10	1,000,498,099	5.703	− 0.265	28.9	12.8
LZ4HC -12	956,532,249	5.965	− 0.003	9.2	14.6
LZ4HC -12w	956,132,233	5.967	− 0.0002	8.3	13.4
LZ4opt	956,106,286	5.968	0	8.9	1,484.2

Figure 1.5 shows the per-file compression ratio and speed in the same format as for the Silesia corpus. The files in the Pizza&Chili corpus are divided into four categories according to their nature, so we plot each category separately.

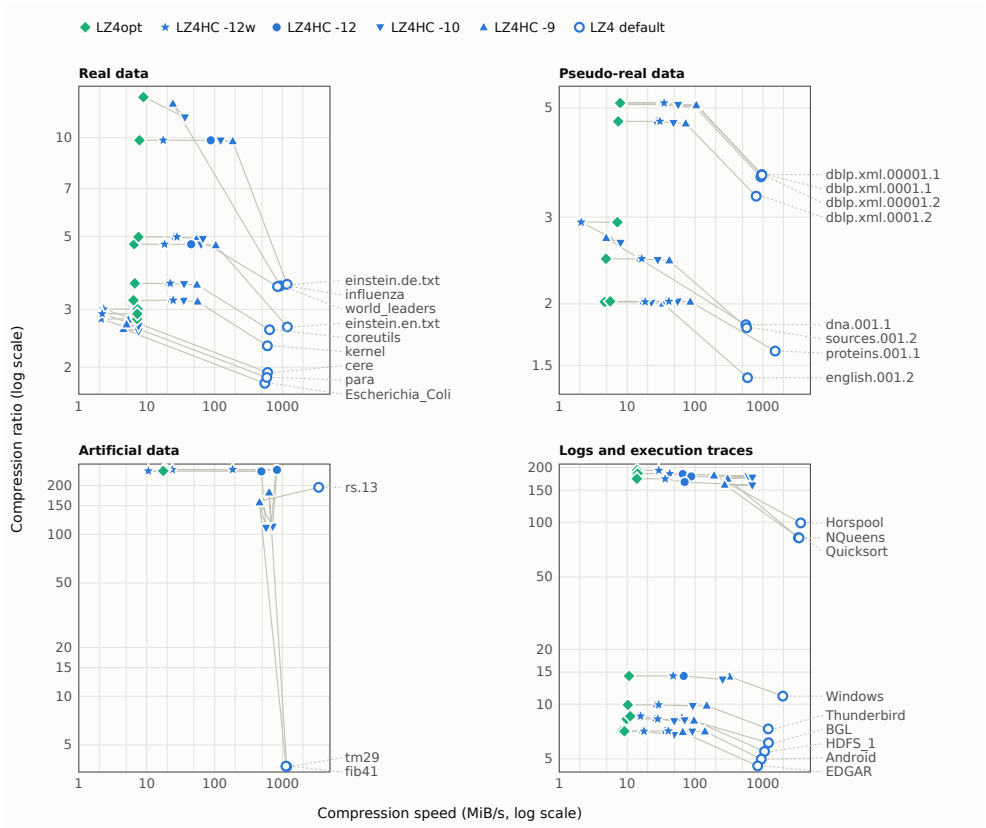


Figure 1.5: Compression speed and compression ratio for each file in the Pizza&Chili corpus

Unlike on the Silesia corpus, LZ4opt showed both a higher compression ratio and a faster compression speed than LZ4HC -12 on some data. Highly redundant data has many match candidates, so LZ4HC -12 spends a long time searching. In contrast, LZ4opt uses a sliding suffix tree and can compute the LPF at each position in amortized constant time regardless of the number of candidates. We believe that this difference causes the speed gap. In addition, for the artificial texts fib41 and tm29, the compression ratio of LZ4 default drops sharply to around 4. We believe that the fast mode' s technique of skipping ahead through regions where no match is found backfires on these strings.

The difference between LZ4HC -12 and LZ4opt varies greatly from file to file. The absolute difference in compression ratio between LZ4HC -12 and LZ4opt was less than 0.001 for 21 of the 29 files. Table 1.4 shows the compression ratios of the top 5 files

by absolute ratio difference (the execution traces Horspool, Quicksort, and NQueens, and the artificial texts fib41 and tm29).

Table 1.4: Comparison of the 5 files with the largest ratio differences

File	LZ4HC -12	LZ4HC -12w	LZ4opt	Ratio difference of LZ4HC -12	Ratio difference of LZ4HC -12w
Horspool	183.94	192.49	193.16	− 9.22	− 0.67
Quicksort	166.25	172.75	173.24	− 7.00	− 0.49
NQueens	178.52	184.58	185.07	− 6.55	− 0.49
fib41	249.94	250.90	250.93	− 0.99	− 0.02
tm29	244.36	245.64	245.64	− 1.27	0.00

Table 1.5 shows the number of matches for the same 5 files. The number of matches is the number of sequences that contain a match. The difference is the number of matches of each method minus that of LZ4opt.

Table 1.5: Number of matches for the 5 files with the largest ratio differences

File	LZ4HC -12	LZ4HC -12w	LZ4opt	Difference of LZ4HC -12	Difference of LZ4HC -12w
Horspool	267,989	235,419	232,886	+35,103	+2,533
Quicksort	247,336	230,017	228,724	+18,612	+1,293
NQueens	158,727	148,152	147,332	+11,395	+820
fib41	5,942	3,424	3,458	+2,484	− 34
tm29	12,480	12,416	12,416	+64	0

For all 5 files, LZ4opt represents the input with fewer matches than LZ4HC -12, which we believe contributes to the higher compression ratio of LZ4opt. With LZ4HC -12w, which widens the optimization range, the number of matches also approaches that of LZ4opt.

For fib41, LZ4HC -12w uses fewer matches than LZ4opt, yet LZ4opt produces the smaller compressed size. The encoded length includes not only tokens and offsets but also the costs of literal bytes and length extension bytes, so minimizing the number of matches does not necessarily coincide with minimizing the encoded length.

With LZ4HC -12w, the ratio difference from LZ4opt shrank by more than 90% for all 5 files, and for tm29 the compressed sizes matched. These results suggest that, for these files, the limited optimization range and the early commit of long matches are the main causes of the difference in compression ratio. However, widening the optimization range can increase the compression time. For example, for Horspool, switching from LZ4HC -12 to LZ4HC -12w increased the compression time from 8.3 seconds to 18.6 seconds.

1.9 Conclusion

We implemented LZ4opt, an encoder that minimizes the encoded length of each block in linear time within the LZ4 format. This makes it possible to check on real data how far the compressed sizes of existing encoders are from the optimum.

On the Silesia corpus, the difference between the highest compression setting of LZ4HC (`-12`) and the optimum was only 282 bytes over the entire input of about 202MiB, showing that almost no room for improvement remains. On the other hand, for some files in the highly repetitive Pizza&Chili corpus, the compression ratio of LZ4opt exceeded that of LZ4HC -12 by up to about 5%, showing that for some data there is still room to reduce the compressed size further.

We initially expected a considerable gap between LZ4HC and the optimum, and hoped that observing this gap would provide clues for improving the compression ratio. We were therefore very surprised that the difference between LZ4HC -12 and the optimum on the Silesia corpus was so small. Although the result differed from our expectations, we consider the confirmation of how little room for improvement remains to be a useful finding in itself for deciding the direction of future development. At least under the conditions of this study, we found that LZ4HC -12 is nearly optimal on the Silesia corpus and that further improvement is largely unnecessary. We believe that LZ4opt will also serve as a benchmark for evaluating how close existing implementations come to the limit when dealing with new kinds of data.

References

[1] Yann Collet, LZ4 - Extremely fast compression, GitHub, 2011
`https://github.com/lz4/lz4`
[2] O3DE, O3DE Documentation: Multiplayer Compression Gem, Web page, 2026
`https://docs.o3de.org/docs/user-guide/gems/reference/multiplayer/mu`

ltiplayer-compression/

[3] The Apache Software Foundation, Apache Cassandra Documentation: Native Protocol, Web page, 2026

https://cassandra.apache.org/doc/latest/cassandra/reference/native-protocol.html

[4] The Linux Kernel Contributors, The Linux Kernel Documentation: zram: Compressed RAM-based block devices, Web page, 2026

https://kernel.org/doc/html/v6.12/admin-guide/blockdev/zram.html

[5] OpenZFS Project, OpenZFS Documentation: zfsprops(7), Web page, 2026

https://openzfs.github.io/openzfs-docs/man/master/7/zfsprops.7.html

[6] Unity Technologies, Unity Manual: AssetBundle compression formats, Web page, 2026

https://docs.unity.com/en-us/engine/6000.0/manual/assets-and-media/assets-managing-runtime/assetbundles-section/creating/compression-format

[7] The Apache Software Foundation, Apache Spark Documentation: Spark Configuration, Web page, 2026

https://spark.apache.org/docs/latest/configuration.html

[8] The PostgreSQL Global Development Group, PostgreSQL 18 Documentation: Client Connection Defaults, Web page, 2026

https://www.postgresql.org/docs/18/runtime-config-client.html

[9] Lempel Abraham, Ziv Jacob, On the Complexity of Finite Sequences, IEEE Transactions on Information Theory, 1976

https://doi.org/10.1109/TIT.1976.1055501

[10] Ziv Jacob, Lempel Abraham, A Universal Algorithm for Sequential Data Compression, IEEE Transactions on Information Theory, 1977

https://doi.org/10.1109/TIT.1977.1055714

[11] Weiner Peter, Linear Pattern Matching Algorithms, 14th annual symposium on switching and automata theory (SWAT), 1973

https://doi.org/10.1109/SWAT.1973.13

[12] Ukkonen Esko, On-line construction of suffix trees, Algorithmica, 1995

https://doi.org/10.1007/BF01206331

[13] Fiala Edward R., Greene Daniel H., Data Compression with Finite Windows, Communications of the ACM, 1989

https://doi.org/10.1145/63334.63341

[14] Larsson N. Jesper, Extended Application of Suffix Trees to Data Compression,

Proceedings of the data compression conference (DCC), 1996

https://doi.org/10.1109/DCC.1996.488324

[15] Leonard Laurentius, Inenaga Shunsuke, Bannai Hideo, Mieno Takuya, Sliding Suffix Trees Revisited, arXiv:2307.01412, 2023

https://arxiv.org/abs/2307.01412

[16] Sebastian Deorowicz, Silesia compression corpus, Web page, 2003

https://sun.aei.polsl.pl/~sdeor/index.php?page=silesia

[17] Pizza&Chili Corpus, Repetitive Corpus, Web page, 2010

https://pizzachili.dcc.uchile.cl/repcorpus.html